Greenwash-RAG: Evidence-Grounded, Cross-Year Tracking of Sustainability Claims in SGX Reports

AI6127 Group Project Report, Group 6

Member One	Member Two	Member Three
G2XXXXXXA	G2XXXXXXB	G2XXXXXXC
member1@e.ntu.edu.sg	member2@e.ntu.edu.sg	member3@e.ntu.edu.sg

Member Four	Member Five	Member Six
G2XXXXXXD	G2XXXXXXE	G2XXXXXXF
member4@e.ntu.edu.sg	member5@e.ntu.edu.sg	member6@e.ntu.edu.sg

Abstract

We build a retrieval-augmented system over 240 sustainability reports of 56 SGX-listed issuers (2020–2025, 10,635 pages, 3.5 M words) and ask two questions. **RQ1:** given one sentence and a candidate sustainability aspect, does evidence retrieved from the company's own reports change the action label a model assigns under the A3CG scheme (implemented / planning / indeterminate / none)? **RQ2:** when a report makes a dated commitment, what do the company's later reports say became of it? The knowledge base is every statement of the parsed reports (108,376 chunks); the retriever is BM25 plus dense retrieval fused by reciprocal rank fusion and re-ranked by a cross-encoder, with company and year filters; a temporal claim graph and a bounded agentic planner are the two retrieval innovations; one shared classifier with citations serves every arm. On a self-annotated set of 1,000 sentences (six annotators, Fleiss κ 0.81 on the shared pilot), retrieved evidence moves majority-vote macro-F1 by at most 0.04 on the development split, never significantly (paired McNemar $p \geq 0.21$); the claim graph matches hybrid retrieval at a tenth of the retrieval latency, and statement-level evidence beats 200-word passages for cross-year evidence ($p = 0.043$). For RQ2 we extract 1,273 validated dated targets, follow 373 checkable commitments through later reports, and label each statement before a fixed rule names the outcome. Against a gold set of 373 commitments (137 double-annotated by the group, κ 0.84; the rest by two calibrated model annotators of different families), the graph's trajectory finder, plain hybrid retrieval and an agentic planner are indistinguishable on the development split (vote accuracy 0.79–0.81) and on the test split (0.71–0.78 under the judge of record, paired $p \geq 0.30$); verdicts are right 85–94% of the time and the system gives a verdict on 14–28% of the commitments a reader cannot settle. A judge that reads the company's own account in other words as the commitment gains 11 to 14 of 250 test promises on every arm, short of significance. Human-verified, about half of the dated commitments in these reports were met, one in eight was missed or re-dated, and a third cannot be settled from the text. RQ1's test split is unread; every number carries its run identifier.

1 Problem Definition

The user task. SGX requires every mainboard issuer to publish an annual sustainability report. Analysts, investors, regulators and journalists ask questions of the form "Company X committed to an emissions target in 2021; was it delivered, and on which pages is the evidence?" Answering by hand means reading three or four years of templated ESG prose. Our system takes a company and either a sentence with a candidate aspect or a dated commitment, retrieves the company's own report text across years, and returns a page-cited answer: for a sentence, an A3CG action label with a rationale and citations; for a commitment, an outcome with the statements it rests on. Sample inputs are the five timed queries of Table 4 and the case studies of Section 6.5. The output is assistive screening with sources, never a verdict that a company is greenwashing (Ong et al., 2025; Calamai et al., 2025).

RQ1: does evidence change the label? An item is a tuple (s, a, c, y, p): a sentence s from the report of company c in year y on page p, with a candidate

aspect a (for example *Scope 3 emissions*). The classifier $f(s, a, E)$ returns a label in {implemented, planning, indeterminate, none}, where *none* means the proposed aspect is not a valid aspect–action pair, following the A3CG scheme (Ong et al., 2025). E is the evidence condition: nothing (the A3CG setting), the neighbouring sentences on the same page, up to k statements retrieved from the same report, or up to k statements retrieved from the company's *other* years. The question is Δ macro-F1 of each condition over sentence-only, measured on the same items with a paired test, and whether a structured retriever (a claim graph) or a planning retriever (an agent) reaches a different Δ from plain hybrid retrieval at the same evidence budget.

RQ2: what became of dated commitments? A *promise* is a statement by company c in report year y_0 that names a target (metric, quantity, unit, baseline) and a deadline year $d \geq y_0$. The system must return an outcome in {achieved, failed, unclear, disappeared} from the company's reports of years $y_0{+}1 \ldots d{+}1$, together with the statements that justify it. *Failed* covers a deadline that passed without the target being reported as met, whether the report admits the miss or moves the date; *unclear* is the "not enough information" class (Thorne et al., 2018); *disappeared* means no later statement about the target or its metric. The taxonomy extends Jiang et al. (2025), who code emissions targets as achieved, failed or disappeared.

Scope. Both questions are answered from the issuer's own reports; no external news or regulatory evidence is used. Raw reports are copyrighted and are never redistributed; the knowledge base we share is derived text with provenance. Table 1 maps the questions of the assignment brief to the sections that answer them.

Brief item	Where
Q1 corpus: gathering, cleaning, chunking, storage; task and sample queries; documents, chunks, words, types	§3.1, Tables 2, 4
Q2 UI; five queries with results and timing	§4.9, Figure 2, Table 4
Q3 retrieval innovations with examples; P@K, R@K, MRR, nDCG	§4.3–4.10, §4.8; rank-aware metrics **[Pending Result]** (§3.4)
Q4 approach vs. state of the art; preprocessing; 1,000-record set with IAA $\geq 80\%$; P/R/F; RAG metrics; latency, throughput, cost, scalability	§4.6, §4.7, §3.2, §5.1–5.5
Q5 downstream innovations with ablations	§3.3, §5.1, §5.4, §6.1–6.3
Names and matriculation numbers; data and code links	first page; §3.6

Table 1: Where each item of the assignment brief is answered.

2 Related Studies

Aspect–action analysis of ESG text. A3CG (Ong et al., 2025) annotates 2,004 statements from SGX sustainability reports (2017–2022) with aspect spans and an action label in {implemented, planning, indeterminate}, where *indeterminate* covers hedged intent, bare disclosure and responsibility statements, the classes most exposed to greenwashing. Its supervised models reach pair-level F1 of 67–71 on the full set and 41–48 on unseen categories; action classification is harder than aspect extraction everywhere, and the paper reports no inter-annotator agreement. The same group's follow-up, COGLM (Braun et al., 2026), adds contrastive, ordinal and gated objectives on LoRA adapters and sets the model-side state of the art (72.4 full / 67.6 seen / 49.0 unseen). Both classify one isolated sentence; A3CG's own error analysis names *ellipsis* (missing context) and *unattributed statements* as the dominant failure modes, which are exactly what company-scoped retrieval could supply. That is our RQ1.

Promise verification and greenwashing detection. ML-Promise (Seki et al., 2025) labels promise identification, actionable evidence, clarity and verification timing over ESG reports in five languages, locating evidence *within the same document*; the English portion is about 200 instances and there is no cross-year dimension. EmeraldMind (Kaoukis et al., 2025) is the closest system: a vector store and an ESG knowledge graph (53,748 entities from 37 reports) behind a RAG pipeline that returns greenwashing / not / abstain with an evidence-grounded justification, with hybrid retrieval the best variant; its claims are semi-synthetic and it has no temporal dimension. Florstedt et al. (2025) use specificity and internal claim substantiation as greenwashing hints over German DAX reports. The survey of Calamai et al. (2025) finds that no dataset of verified greenwashing cases exists and recommends decomposed pipelines with

traceable reasoning and human oversight, which is the architecture we adopt. ESGBench (George and Saji, 2025) shows how hard numeric and table grounding in ESG PDFs is for RAG (exact match 21%). Jiang et al. (2025) code 1,541 CDP emissions targets as achieved, failed or disappeared and find that 31% of firms' targets disappear; no SGX study checks whether earlier commitments were met. Our RQ2 fills that gap with a four-class taxonomy and page-cited evidence.

Retrieval and graphs. Our retriever follows the course design: BM25 (Robertson and Zaragoza, 2009), dense embeddings, reciprocal rank fusion (Cormack et al., 2009) and a cross-encoder reranker (Reimers and Gurevych, 2019; Wang et al., 2020), inside the RAG contract of Lewis et al. (2020). Community-summary GraphRAG (Edge et al., 2024) targets global sense-making questions and is expensive to index; our queries are local and structured, so we build a small temporal claim graph instead. Zhou et al. (2025) find that graph-only methods lose to vector RAG on specific questions and that the original text chunks must be kept, while graph structure pays off on multi-hop questions; our graph arms therefore always return statements with their page, and a fused arm is kept. Ferrazzi et al. (2026) find that agentic RAG is not universally better than enhanced RAG, costs 3.3× the input tokens, and is worse than a re-ranker at final document selection; our agent plans and rewrites queries but leaves selection to rank fusion or the re-ranker. For canonicalising free-text aspects we follow CESI (Vashishth et al., 2018), EDC (Zhang and Soh, 2024) and Peeters et al. (2025).

Verification, abstention and evaluation. FEVER (Thorne et al., 2018) established the *not enough information* class and showed it is the hard one; ClaimDecomp (Chen et al., 2022) derives a verdict from yes/no sub-questions; Wen et al. (2025) survey abstention as a way to cut unsupported output. Our RQ2 judge labels each statement with three sub-questions and lets a fixed rule name the outcome, defaulting to *unclear*. Answer-level RAG metrics follow RAGAS (Es et al., 2024); majority voting over repeats follows self-consistency (Wang et al., 2023); systems are compared with the paired McNemar test (McNemar, 1947; Dietterich, 1998), and agreement with Fleiss' and Cohen's κ (Fleiss, 1971; Cohen, 1960).

What is open. Nobody has asked whether retrieved report-internal or cross-year evidence changes the A3CG action label; no ESG benchmark follows a commitment across report years; and A3CG's report list is unpublished, so a longitudinal SGX corpus with page-level provenance is itself an artefact.

3 Experiment Design

3.1 Corpus and knowledge base

Gathering. Reports were discovered through the public sitemap of SustainabilityReports.com (discovery only; its terms forbid bulk download) and fetched from the issuers' own investor-relations sites on the shared `listedcompany.com` platform (229 files; `robots.txt` allows crawlers; at most one request per second per host) and, for 11 pilot files, from company sites or SGXNet. Every candidate is a row of `kb/manifest.csv` with its URL, fetch date and the rule that kept or rejected it.

Cleaning. Eleven ordered filters, all auditable from the manifest: year outside 2019–2026 in the URL; SHA-256 duplicates; fewer than 500 words of text layer (scanned); under 8 pages (cover letters); more than 20% CJK or Thai characters (non-English editions); AGM notices and circulars; full annual reports that do not mention sustainability on page 1; GRI indexes, TCFD supplements, policies and frameworks by filename and title page; a positive test that the first ten pages name a sustainability or ESG report; the reporting year taken as the calendar year in which the period ends; one document per company-year (the longest). A random sample of 15 kept reports was checked by hand; all were genuine sustainability reports of the named issuer with a plausible year. Three reports downloaded from SGXNet should be replaced by company-hosted copies before any redistribution, since SGX's site terms may forbid extraction for analysis.

Parsing and chunking. PDFs are parsed page by page, ligatures and footnote digits repaired, and the text split into statements with the same sentence splitter the annotation set used, so every one of the 1,000 evaluation sentences matches a parsed statement inside its own report. Tables flatten into one statement per row or cell. The retrieval unit is the *statement*: every statement of five words or more is one chunk, prose, table rows and metric lines alike, because a question about a number, or about what

reports (documents)	240
issuers	56
report years	2020–2025
reports per year	38 / 38 / 40 / 43 / 43 / 38
issuers with ≥ 3 years (all six)	41 (29)
pages	10,635
words in the PDF text layer	3,475,125
parsed statements	110,314
words / types (unique lower-cased word forms)	3,401,540 / 29,309
words per statement, mean / median / p90	30.8 / 23 / 47
chunks, statement store (default; ≥ 5 words)	108,376
chunks, passage store (ablation; $\approx$ 200 words)	18,839
store hashes (statement / passage)	`d8a62bfe40c6` `c80d0ab23191`

Table 2: The corpus (Question 1). Both chunk counts exceed the 10,000-passage floor; the word count exceeds the 100,000-word floor thirty-fold.

became of a target, needs exactly the rows a prose filter would drop (Section 4.8 measures that cost). A second store of 200-word passages (one sentence of overlap, never crossing a report) is kept as the granularity ablation. Each chunk carries {company, year, page, file, statement ids} and is indexed with a contextual header "Company (Year), page N: …" that helps both BM25 and the encoder.

Storage. A store is a directory with `chunks.jsonl`, L2-normalised float32 vectors (`text-embedding-3-small`, 1,536 dimensions) and a manifest whose hash over the chunk texts is written into every run; the two stores are packaged as zip files (557 MB and 104 MB) and shared inside the group. The same chunks are mirrored into Postgres with pgvector (HNSW cosine index, b-tree on company and year); on development queries the file and database backends return the same top-5 lists. The parsed statements, reports and the claim graph live in DuckDB.

3.2 Evaluation sets

RQ1: gw1000. 1,000 sentences from 52 issuers' 2023–2025 reports, each with one candidate aspect and its A3CG category (or none), page-level provenance and the neighbouring sentences. Sentences were pre-annotated by three LLMs (used only to propose aspects and for label-noise analysis; they agree with the human majority 0.79–0.80 and are never a system input), then labelled by six annotators under an A3CG-aligned guideline. The 50 pilot rows carry all six labels (observed agreement 0.861, Fleiss κ 0.809), the other 950 one label each, one tie adjudicated; 34 rows shown as *no aspect* received an action label and had their span filled by review (flagged, reported separately). Labels: implemented 417, planning 191, indeterminate 222, none 170; no duplicate sentences. Split: dev 331 / test 669, stratified by row type, label and category, pilot rows pinned to dev. The file `kb/annotation/eval.xls` is the tabular export the brief asks for; the frozen snapshot `gw1000.human_majority.3e18cc14e648.jsonl` carries its hash into every run manifest.

RQ2: rq2promises. Every checkable promise (Section 4.8) carries a settled outcome. 137 were labelled on a blind two-annotator sheet by the group (agreement 0.898, Cohen's κ 0.84, adjudicated to a final label). The remaining 237 were labelled by two model annotators of different families (Claude Fable, Claude Opus) run as sub-agents under the same sheet guide, reading exactly the cells the humans read, blind to each other and to the system, after calibration on fresh stratified slices of 40 human rows: 0.875 and 0.925 against the adjudicated human final (κ 0.81 / 0.88), where the two humans agree 0.95 with each other on the same slice; two instances of one model agreed 79 times in 80, which is why the second annotator was moved to another family. The pair agreed on 221 of 237; the 16 disagreements went to a third pass; one row settled as *not a promise* (an extraction error) leaves the set. Gold over 373: achieved 206, failed 68, unclear 86, disappeared 13 (32 companies, deadlines 2021–2025). Split: dev 123 / test 250, stratified by outcome and label source, the 75 promises the judge prompt was tuned on pinned to dev; dev holds 22 human-labelled rows, test 115.

3.3 Evidence conditions and arms

One classifier, one prompt, one budget ($k = 3$ spans of at most 800 characters) for every RQ1 arm; systems differ only in the evidence they hand it (Table 3). The harness drops any returned span that *is* the target sentence and counts the drops, so no arm can score by re-reading what it classifies. For RQ2 the same `retrieve(query, k)` call serves every system under the `promise_window` condition: the query carries the company, the year window ($y_0{+}1$ to $d{+}1$), the metric and the promise record, and at most $k = 20$ statements come back for the shared judge.

condition	evidence the classifier sees
`sentence_only`	nothing (the A3CG setting; the baseline for every Δ)
`local`	neighbouring sentences of the same page
`same_report_rrf`	k statements retrieved from the same report (hybrid)
`cross_year_rrf`	k statements from the same company, other years (hybrid)
`cross_year_graph`	statements reached through the claim graph
`cross_year_fused`	graph first, BM25 over other-year statements fills to k
`cross_year_agentic`	the agent over other years: no evidence unless the sentence has a gap, then only passages verified to close it
`company_agentic`	the same over every report of the company, starting beside the sentence
`promise_window`	RQ2: ≤ 20 statements from later reports, read by the judge

Table 3: Evidence conditions. The first four are served by every system; the graph and agentic conditions by the system that owns them.

3.4 Metrics

Classification. Macro-F1 over the three action labels on rows whose gold label is an action (331 on dev), with micro-F1, accuracy, per-label precision / recall / F1 and confusion matrices; detection F1 for *none*, because evidence can talk a model into inventing an aspect. *Stability.* Three uncached repeats per condition: mean $\pm$ sd, the items whose label changed between repeats, and the majority vote. *Comparison.* Re-running an identical configuration does not reproduce its score (three sentence-only votes on the same items: 0.7397 / 0.7498 / 0.7580), so arms and systems are compared only with the paired McNemar test on majority votes over the same items, with pooled flips wrong→right / right→wrong and a bootstrap 95% interval. *Answer-level RAG metrics.* Faithfulness, answer relevance, context precision and context relevance as defined by RAGAS, implemented in-house so every intermediate judgement is kept, scored by the same model and calibrated against a human on a blind sheet. *RQ2.* Accuracy, κ, macro-F1 over the four outcomes, the share of gold-unclear promises given a verdict (over-commitment), the share of settled promises left unclear (under-commitment), the precision of the verdicts given, and a bootstrap interval; the judge version is a run parameter, and a judge can be re-run on another run's stored evidence so that judges are compared on identical statements. *Performance.* Retrieval, classification and end-to-end latency (p50 / p95), items per second, input and output tokens, and USD per 1,000 items at 0.20 / 1.20 USD per million input / output tokens. *Rank-aware retrieval metrics.* The harness computes P@5, R@10, MRR and nDCG@10 from pooled relevance judgements (`gw-eval -task retrieval`, with a pooled judging sheet over every mode's top-k); the judgements have not been collected, so these numbers are **[Pending Result]**.

3.5 Protocol

Dev for tuning, test opened once (RQ2's was read once per arm with the comparisons frozen beforehand, Section 6.3; RQ1's has not been read); repeats $\geq$ 3 with the response cache bypassed; the same classifier, prompt, model, reasoning effort and budget for every arm, so any change re-runs every arm; the gold label is stripped before an item reaches a system; every number in this report names the published run (`results/<group>/<name>/`), the snapshot hash and the store hash it comes from. Chat model `gpt-5.6-luna` at low reasoning effort and temperature 0; embeddings `text-embedding-3-small`; re-ranker `ms-marco-MiniLM-L-6-v2` on CPU.

3.6 Resources

Knowledge base, queries and their retrieved results, the evaluation sets, predictions and judge outputs: https://example.com/REPLACE-WITH-DATA-LINK. Source code with a README that explains how to install and run everything: https://example.com/REPLACE-WITH-CODE-LINK. The repository is a `uv` workspace (`src/common`, `src/baseline-rag`, `src/graph-rag`, `src/agentic-rag`, `src/example-rag`, the Streamlit demo in `frontend/`); 109 unit tests run without network access.

4 Methodology

4.1 Architecture

Figure 1 shows the pipeline. A *system* implements two small protocols, `retrieve(query, k)` and `classify(items, evidence)`, and registers an entry point; the harness in `src/common` does everything else. Three systems are compared: the hybrid baseline, the graph system and the agentic

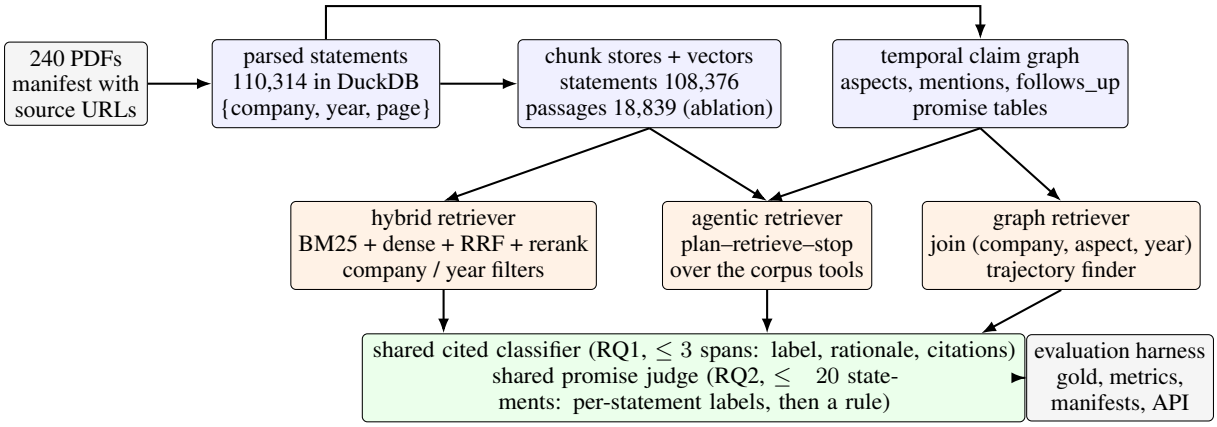


Figure 1: Architecture. Every retriever reads the same parsed statements; every RQ1 arm hands at most three spans to one classifier, every RQ2 arm at most twenty to one judge; the harness owns the data, the gold labels, the metrics and the run manifests.

system, all over the same knowledge base and with the same shared classifier and judge.

4.2 Hybrid retriever (sparse, dense, fusion, re-ranking)

The baseline retriever is built from parts, not from a hosted service: an in-process Okapi BM25 index ($k_1 = 1.5, b = 0.75$) over the contextualised chunk text; a dense index of `text-embedding-3-small` vectors searched by cosine over L2-normalised vectors (pgvector HNSW when the Postgres backend is on); reciprocal rank fusion of the top 50 from each ($k_0 = 60$); and a cross-encoder (`ms-marco-MiniLM-L-6-v2` via fastembed, ONNX on CPU) over the fused top 20. A mode switch (`bm25`, `dense`, `rrf`, `rrf+rerank`) runs the ablation through one code path. Metadata filters come from the condition: the same report, or the same company excluding the target's year, or an explicit year window; the retriever never returns the target sentence.

4.3 Temporal claim graph

Three of the project's needs are joins, not similarity problems: the same company's statements on the same aspect in other years (ESG prose is templated, so nearest neighbours without a company filter are other companies' near-identical sentences, as query Q5 in Table 4 shows); following a commitment forward in time; and detecting a moved baseline or a dropped target. EmeraldMind's graph has no temporal dimension, so a *temporal* claim graph is the defensible contribution. It is stored in DuckDB with networkx for traversal of a company's subgraph: 56 companies, 240 reports, 110,314 statements, 7,503 aspects (734 from the annotated spans and their string matches, 6,769 extracted by an LLM), 29,637 statement→aspect mentions carrying the action label where one exists, and 8,524 `follows_up` edges (statement → later statement of the same company and aspect, with `years_apart` and a `same_text` flag; 28% of the edges join verbatim repeats, a fact about templated reporting the trajectory logic uses). Two gates shaped the build. Page furniture (GRI index rows repeated every year) produced thousands of false chains until a sentence-shape gate on what the aspect extractor sees dropped 23,097 candidate mentions; the gate applies only to extraction, never to retrieval. LLM aspect extraction over 9,320 statements passed its gate on the 1,000 gold sentences (partial-match F1 0.649, category accuracy 0.903, false-positive rate on no-aspect sentences 0.264) and ingestion rejects any span that is not a verbatim substring. Canonicalisation of the free-text aspects did *not* pass its gate: on a clean test set of 81 pools and 123 gold pairs, embeddings with complete-linkage HAC reached pairwise precision 0.467, and embeddings proposing with an LLM verifying under written rules 0.597 [0.49–0.70], against a gate of 0.80; the graph therefore keeps morphological normalisation only (7,503 → 7,319 ids), and 79% of the extracted aspect nodes occur once. The graph arms return statements with their page, never bare nodes: `cross_year_graph` joins the item's aspect to the company's other-year statements that mention it (coverage 0.40 on dev, 9 ms); `cross_year_fused` fills to k with BM25 over the company's other-year statements (coverage 0.89, 20 ms).

4.4 Agentic retriever

The agentic system is a bounded planner over the corpus tools of Section 4.5, one policy per question, behind a fence in code: a planner that names another company, a year outside the window or the sentence being classified gets nothing, and the agent never names a label or an outcome. It reads the statement store only, never the claim graph. For RQ1 (`cross_year_agentic` over the company's other years, `company_agentic` over every report of the company) the first planner call is a *gate*: does the sentence have a gap another passage could close, an unresolved reference, an unclear actor, an aspect it cannot read? No gap, no evidence, and the classifier reads exactly what `sentence_only` reads. With a gap the planner writes it as a question and gives keyword queries, a *verifier* call says which of the passages found answer the question, and only those reach the classifier, in cross-encoder order against the question; the verifier is told that a passage saying a plan was later carried out does not count, since that does not change an A3CG label. For RQ2 (`promise_window`) hop 0 makes no model call: a budget per report year and the neighbours of the best hits, the trajectory finder's rules over the statement store (with `AGENTIC_MAX_HOPS=0` this is the *fixed-budget* arm); then one fixed search for a moved or dropped deadline, so that a statement that looks like success does not end the search for the contrary one; then up to two planner hops that read what has been gathered and ask for what is missing through `search`, `around` (the statements beside a hit) and `page`, their finds keeping 8 of the 20 slots. Every planner decision is traced and the planner's tokens are charged to the prediction, so cost is end to end. An earlier version of the arm (rank fusion over the planner's queries, on the passage store) spent 3.2× the plain search and 839 of its 879 spans were what hop 0 had found anyway; the gate and the verifier replace it, following Ferrazzi et al. (2026), who find agents worse than a re-ranker at final selection.

4.5 The corpus as tools; one protocol for both questions

So that an agent plans over exactly what the fixed baseline uses, the corpus is exposed as a small tool set: `search(text, company, year_from, year_to, exclude_year, k, mode)` in four modes, `get`, `neighbours` (same page, report order), `page`, `reports(company)` and `companies()`; the graph corpus adds `aspects`, `follow_ups` and `related_targets(company, metric)`. The hybrid baseline is one `search` call; an agentic retriever is a plan over the same calls; both end in the same `retrieve`, and the same `retrieve` serves RQ2 under `promise_window`, so a teammate's system takes part in both questions without changing its interface.

4.6 Shared classifier (RQ1)

Choice, against the state of the art. The supervised A3CG models and COGLM are the strongest published classifiers, but they read one sentence and cannot be conditioned on retrieved evidence without retraining, and RQ1 needs the *same* classifier under every evidence condition. We therefore use one zero-shot LLM classifier (`gpt-5.6-luna`, low effort, temperature 0) with a prompt that encodes the A3CG guideline, including the three *indeterminate* sub-rules, and reads up to three evidence spans tagged E1…E3. On A3CG's own released test splits with the gold aspect given it scores macro-F1 0.748 (0.714 / 0.723 averaged over the seen / unseen folds), a reference rather than a like-for-like comparison, since our items carry provenance and theirs do not. *Grounding.* The classifier returns a label, a one- or two-sentence rationale and the citations it rests on (the target, the context, or Ei), resolved to company, year and page; asking for them costs no accuracy (paired p 0.27–1.00 against the one-word version) and makes faithfulness measurable. *Self-consistency.* Each condition runs three times with the cache bypassed; the majority vote is the reported label and the spread is the noise floor.

4.7 Preprocessing

The text is PDF-extracted English prose, so no microtext normalisation is needed; the preprocessing that mattered was elsewhere. Broken ligature glyphs (a *ti* rendered as one unreadable character in *consumption*) and footnote digits fused to words ("incidents13") are repaired; multi-column tables flatten into long statements and are kept, because result lines live there; statements are stored in report order so neighbours and pages can be read around a hit; every chunk is prefixed with its company, year and page for indexing. The one filter we removed is the lesson of Section 4.8: a sentence-shape filter that improves aspect extraction removes a third of the statements of five words or more and most of

the lines that report results.

4.8 Promise pipeline (RQ2)

Extraction (knowledge-base construction, done once). Candidate statements name a year not before their report year and contain a commitment cue (3,164). An LLM extracts the target span, metric, quantity, unit, baseline year, deadline year, kind (quantitative / milestone / qualitative) and a recurring flag; mechanical validation rejects a span that does not occur verbatim and a deadline before the report year. Two extractors were run: the project's small model in-repo (1,462 validated targets, USD 0.83) and Claude sub-agents in 12 shards (1,273); statement-level agreement 0.905, κ 0.79; the stronger extraction is the store, the in-repo one the reproducible comparison. Restatements of one commitment across years are merged (1,273 targets $\rightarrow$ 880 promises) and same-metric commitments with another deadline or quantity are linked as `related`. A promise is *checkable* if it was stated before its deadline, the deadline is $\leq$ 2025 with a report for that year or the next, and the metric names something: 374 (32 companies; 198 quantitative, 107 milestone, 69 qualitative; 161 recurring annual targets).

Evidence. The first version read the top six statements of the deadline-year reports under rank fusion of BM25 over every statement and dense retrieval over the sentence-shaped statements only. The final *trajectory finder* ranks every statement of every later report with both rankings complete, takes a budget per report year (6 from the first report at or after the deadline, 3 from the next, 2 from each earlier one), adds the statements before and after the two best hits of the deadline report (a target table parses into one statement per cell and the result sits beside the target), adds up to three linked targets from the promise graph, and shows everything in report order with "directly follows E3" marks: 11.6 statements per promise from 2.0 report years. Under `promise_window` the hybrid baseline instead returns its top 20 inside the same window.

Judge. Version 1 asked the model to name the outcome from the evidence. Version 2 asks three questions per statement (is it about this very target, its metric only, or something else; does it state the deadline year's result as met or missed; does it move the target to a later or no date) and one per promise (is it an annual target renewed every year), and a fixed rule names the outcome: a *met* and no *missed* on a this-target statement $\rightarrow$ achieved; otherwise a later date on a non-annual target $\rightarrow$ revised; otherwise a *missed* $\rightarrow$ failed; otherwise any this-target or same-metric statement from the deadline year on $\rightarrow$ unclear; otherwise disappeared, with the calendar enforced (a report before the deadline year cannot state a miss). A reply that does not label every statement is unclear; the outcome is the strict majority of three uncached runs, and no majority is unclear. *Revised* is reported as *failed*: the deadline was stated, and it passed without the target being reported as met (Section 6.1 gives the reason). A third version (*v3*), written after the development errors were read, keeps the design and adds three labels the rule may use, each of which can be switched off: `proxy`, the company's own account of the same matter in other words ("no significant fines" for a target of zero fines); `partial`, a commitment stated to be part delivered, which counts as a miss under the convention above; and `figure`, the deadline year's number compared with the target in code, only for a target stated as a level in the same unit. It shows 1,000 characters of each statement instead of 500. `-judge v3` selects it, v2 stays the default, and `-evidence` re-judges an earlier run's stored statements, so two judges are compared on identical evidence. v3 has no human check of its own.

Human checks. Blind sheets, one row per promise with the statement, the extracted fields, the related targets and the evidence tagged E1, E2, ... with year and page; the system's proposal is kept in a key file. Rows are sampled with a floor per proposed outcome and at most four per company, so scores are weighted back to the pool with bootstrap intervals. Three sheets were filled: 75 promises on the first system (one annotator working through an AI-drafted first pass), 60 held-out promises on the final system (same annotator, after the judge was tuned on the first sheet), and 150 promises for two annotators (the 60 plus 90 never checked), of which 137 were settled and form the human part of the gold set.

4.9 Serving, the UI, and five timed queries

`gw-serve` exposes one FastAPI application with one router per system, so a front end switches systems with a parameter: `POST /classify` (text, company, year, aspect, condition) returns the label, rationale, citations and the evidence shown;

`POST /retrieve` returns ranked statements with company, year and page; `GET /trajectory` returns a company's statements on an aspect over the years with their labels; `GET /graph/subgraph` returns a company's aspect subgraph; `GET /runs` lists evaluation runs. Every response that shows text carries its provenance, which is the detail the brief asks a UI to design around.

The user interface is a Streamlit page over the same code the evaluation runs (`frontend/`, Figure 2): the sidebar switches between the two questions and shows each one's definition, the system, the evidence condition, k, the number of model runs to vote over and the setup (model, prices, knowledge-base counts and hash, git revision). The page draws one item of the question's development split (another at a click, or by id), or takes a query of the user's own, and runs it through the chosen system step by step with a live timer per step: loading the system, retrieval, then the shared classifier (RQ1) or the shared judge (RQ2), once per run. The answer is shown as correct or wrong against the gold with the rationale and the sources it cites, every evidence span with where it came from, the retrieval, generation and end-to-end time, tokens and cost, the gold's own context (annotators, agreement, pre-annotations, or both RQ2 annotators' notes), and any cited page rendered from the raw report; for RQ2 the evidence is a timeline by report year with the judge's label on each statement. Every run lands in a session log with a CSV download. It follows the harness's rules: the system never sees the gold, the target sentence is dropped from the evidence, the shared prompts and default k are used, and cached model responses are off by default so a timing is a real timing.

Table 4 runs five queries that stand for the user task through the corpus tools in every mode: a cross-year aspect lookup, a promise follow-up inside its deadline window, an annual target and its results, a milestone with named sites, and a generic query without a company filter. BM25 answers in 2–4 ms inside a company filter and 20 ms over the whole store; dense and hybrid add about 20 ms when the query vector is served from the response cache (an uncached query adds the embedding round trip, 190–730 ms p50 in the evaluation runs); the cross-encoder adds 100–550 ms on CPU. The re-ranked lists are the most readable: Q3 returns the annual target sentence of each year in order, Q4 the sentence that reports two of the four promised sites, Q2 the table cell with the 89% certification figure beside the 2021 target sentence. Q5 shows why the company filter exists: without it the nearest neighbours of a net-zero commitment are other issuers' near-identical sentences and national targets. The graph's joins on the top hits answer in 4–13 ms. Full results are in `report/queries/`.

4.10 Retrieval innovations and the queries they fixed

Beyond the course pipeline, five changes mattered, each for a specific failure. (1) *Contextual headers*: prefixing each chunk with company, year and page gives BM25 and the encoder the company name a bare table row lacks. (2) *Company and year filters, and the cross-year window*: templated prose makes other companies' sentences the nearest neighbours (Q5); every RQ1 arm filters by company and excludes the target's year, every RQ2 arm reads only the window after the promise. (3) *The statement as the unit*: three 200-word passages from other years are worse evidence than three statements ($p = 0.043$, Section 5.3), and the graph's edges point at statements, so the comparison is on one unit. (4) *Every statement in the dense index*: the first promise evidence fused a complete BM25 ranking with a dense ranking that covered only the 71,342 sentence-shaped statements; RRF with $k_0 = 60$ puts a statement that is first in one list (score $1/61$) below one that is thirtieth in both ($2/90$), so result lines that ranked first to third on BM25 were cut, and only 1.6% of the evidence shown came from outside the dense index. Table 5 lists eight answer statements found by hand; with both rankings complete, a budget per report year and neighbours, six of the eight are shown and the human-verified *unclear* share of commitments fell from 49% to 31%. (5) *The graph join and the trajectory finder*: an exact join on (company, aspect, year) answers in 9–20 ms what the hybrid path answers in 190–730 ms, and ordered, budgeted reading of later reports lets the judge read 11.7 statements instead of 20 at the same accuracy (Section 6.2).

5 Results for RQ1

5.1 Does evidence change the label?

Table 6 and Figure 3 give every arm on the 331 development items with the cited classifier and three repeats (runs `baseline/statement-hybrid-cited`, `graph/graph-arms-cited`,

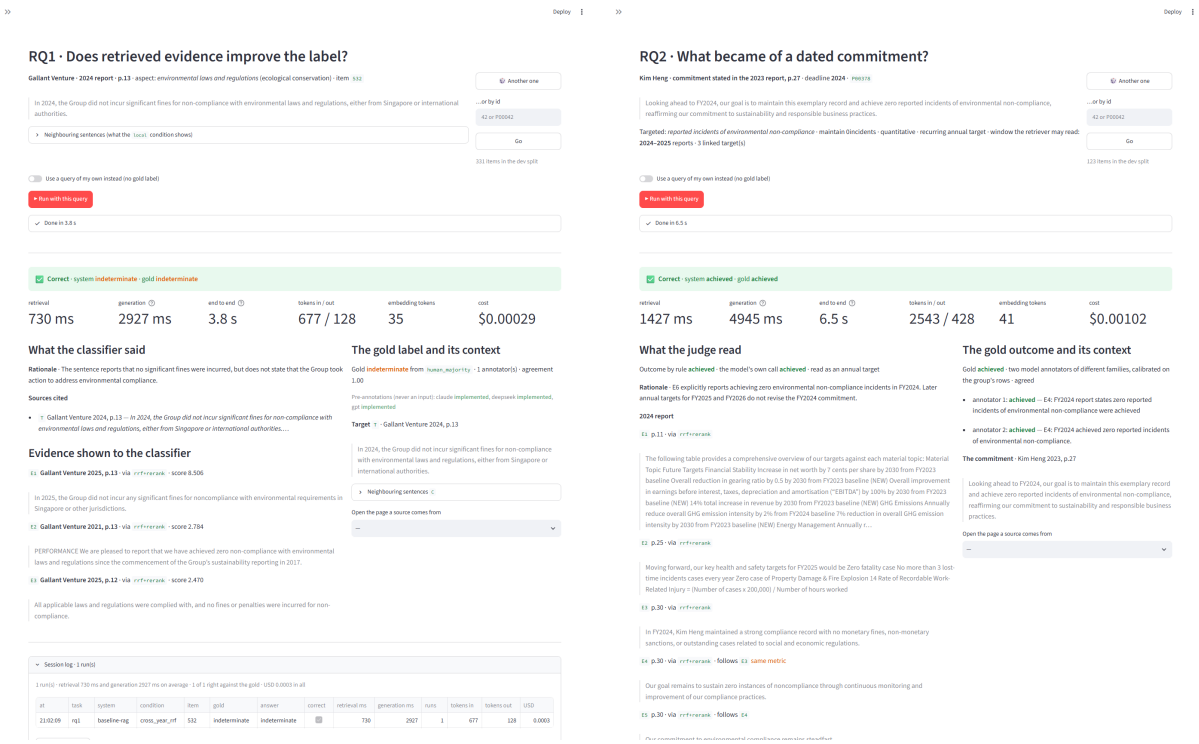


Figure 2: The demo front end (`frontend/`, Streamlit). Left: an RQ1 development item run through the hybrid baseline under `cross_year_rrf`; the label against the gold, the timings, tokens and cost, the rationale with its citations, and the three evidence spans with their source. Right: an RQ2 promise run under `promise_window`; the outcome against the gold and the evidence as a timeline by report year with the judge's label on each statement.

`baseline/passage-hybrid-cited`, all on snapshot `3e18cc14e648`). Three independent runs of the no-evidence configuration give votes of 0.7397, 0.7498 and 0.7580 (pairwise $p = 1.00$ and 0.77): that spread of 0.018 is the noise floor. Against it, the best evidence arm, hybrid retrieval from the company's other years, adds 0.037 to its own sentence-only vote with $p = 0.21$; graph plus BM25 fused adds 0.017 with $p = 1.00$; same-report evidence and the neighbouring sentences add nothing. **Retrieved evidence moves the vote by at most 0.04, never significantly, and the graph is not distinguishable from plain hybrid retrieval** (best hybrid against best graph: $+6/-11$ items, $p = 0.33$). The per-label view (Table 7) shows where the small gain sits: *indeterminate* recall rises from 0.50 to 0.57 under cross-year evidence while *implemented* and *planning* hardly move, which is the direction A3CG's error analysis predicts for missing context. The price is detection: the *none* F1 drops from 0.766 to 0.690–0.739 when evidence is shown, so evidence occasionally talks the model into a pair the annotators rejected.

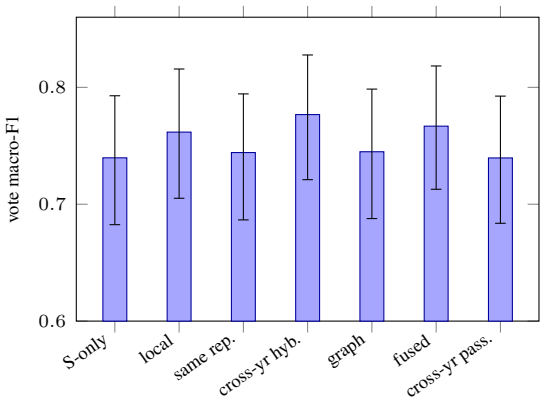


Figure 3: RQ1 majority-vote macro-F1 per arm with the bootstrap 95% interval (dev, 331 items). Every interval overlaps every other; only the paired test can separate arms, and it does not.

5.2 Why so little: the classifier rarely uses what it is given

The classifier cites a retrieved span in 9% of items with same-report evidence and about 1% with other-year evidence from either retriever. The RAG-metric judge (Table 8) finds the rationales faithful (0.96–0.98 of their atomic statements supported by the sources shown) and a minority of the re-

	query (filter)	BM25 ms	dense ms	hybrid ms	+rerank ms	top hit after re-ranking
Q1	Scope 3 greenhouse gas emissions reporting (Gallant Venture, 2022–2025)	3.8	19.2	23.3	571.6	2023 p.26 "Disclose Scope 1, 2, and, if appropriate, Scope 3 greenhouse gas (GHG) emissions and the related risks."
Q2	100% ISPO certification of mills and estates (Indofood Agri, 2021–2025)	2.3	20.5	23.5	430.0	2024 p.43 "Sustainability certification of all estates' hectarage is ISPO certified . . . 89%"; second hit 2021 p.11 "we aim to have 100% of our mills and estates ISPO certified by 2023."
Q3	electricity consumption per square foot target (Old Chang Kee, 2021–2025)	2.9	19.3	22.3	130.1	2021 p.14 "Our target is to maintain or lower our electricity consumption (kwh) per square foot in FY2022."; the FY2023 and FY2024 target sentences follow
Q4	solar rooftops installed at distribution centers (ThaiBev, 2023–2025)	2.4	20.5	21.0	180.9	2023 p.169 "In 2023, ThaiBev installed solar rooftops at two distribution centers in Nakhon Ratchasima and Surat Thani."
Q5	commitment to achieve net zero emissions by 2050 (no filter)	20.2	22.4	44.6	125.4	ThaiBev 2025 p.31 "Thailand has committed to achieving Net-Zero greenhouse gas emissions by 2050."; then CSE Global 2025, Centurion 2025

Table 4: Five example queries (Question 2): median of three calls after a warm-up, $k = 5$, statement store `d8a62bfe40c6`, one machine, CPU re-ranker. End-to-end classification of an item, retrieval plus one classifier call, runs at 2.8–3.8 s p50 in the evaluation runs (Table 11).

trieved statements relevant (context relevance 0.37 same report, 0.29 other years; context precision 0.50 and 0.37). A human calibration of that judge on a blind sheet of 186 rows found it strict on faithfulness (agreement 0.976; it never called supported what the person called unsupported) and lenient on relevance by about 1.7× (agreement 0.831; nine passages called relevant that the person did not), so the corrected context relevance is about 0.22 and 0.17. Relevant sentences are often there and still not needed: for labelling what a sentence says, the sentence carries the answer.

5.3 Statements beat passages as cross-year evidence

Table 9 pairs the passage store (18,839 windows of about 200 words over the same statements) against the statement store on the same 331 items. Inside the same report the two units tie; from the company's other years, three passages are worse than three statements ($+18/-7$ flips, $p = 0.043$), and passage evidence lowers *none* F1 further (0.70 against 0.74). One of four comparisons is significant, so this is a direction rather than a settled fact, but it is why the statement is the project's unit and why the passage store is kept only as the ablation.

5.4 The agentic arm

Table 10 reports the agentic system on the statement store with the cited classifier, six arms in one run so that every pair is on the same items (`agentic/rq1-dev-all-arms`, 331 dev items, three repeats). The gate leaves 262 of the 331 sentences with no evidence; on those the agentic and the sentence-only arm read one identical prompt and the votes still flip $+4/-6$, which is the model's own noise. Over all items `company_agentic` is $+11/-8$ against `sentence_only` ($p = 0.65$) and $+10/-14$ against `cross_year_rrf` ($p = 0.54$); `cross_year_agentic` is $+12/-21$ against `cross_year_rrf` ($p = 0.16$). **The agent does not improve the label and costs nearly twice any fixed arm**, because the gate is a call on every item. What it changes is what the classifier receives: under a tenth of the spans, and 18–24% of those are cited against 0.5–1.3% for plain retrieval. Four ablations (no verifier; three hops; cross-encoder selection; the neighbouring sentences kept whatever the verifier says) stay inside the noise. One finding beyond the agent: a text rule frozen before the run ("this target", a leading "it" or "such", a third-party actor; 75 of 331 sentences) marks the sentences that depend on their context, and on those the neighbouring sentences go $+9/-0$ against sentence-only ($p = 0.004$, macro-F1 0.663 to 0.771) while on the other 256 they go $+12/-14$; routing by that rule and leaving the rest on sentence-only composes to 0.7924 at no planner cost. It is one of ten slice comparisons on dev (Bonferroni threshold 0.04) and has not been read on test. An earlier run of the arm on the passage store with the one-word classifier (`agent-rag/passage-hybrid`, 2026-10-

promise → answer statement	BM25	dense	fused@6
ThaiBev solar rooftops → “installed solar rooftops at two distribution centers”	1	no vector	7
Jumbo complaints → FY2022 performance table	1	no vector	9
Gallant Venture GHG → “achieved a 15.4% reduction”	3	no vector	16
Old Chang Kee electricity → “was 61 kwh per square foot (FY2020: 68)”	3	no vector	18
Indofood Agri ISPO → “by 2024 has been postponed”	5	11	7
Old Chang Kee accidents → “Requires improvement as we recorded two cases”	none	10	19
Tiong Woon ERP → “In FY2022, we began integrating”	none	58	–
Indofood Agri pouch → “has been delayed” (2023 report)	1	1	n/a

Table 5: Queries that failed under the first promise evidence (Question 3): the rank of the hand-found answer statement in each ranking and after fusion cut at six (n/a: the sentence lies outside the first version’s deadline-year window). Six of the eight are shown by the final finder; the ISPO sentence ranks eighth among many ISPO statements and the Tiong Woon sentence is not reachable by similarity.

20) pointed the same way: agentic 0.7555 against cross-year hybrid 0.7688 within that run, at three times the tokens.

5.5 Speed, cost and scalability

Table 11 gives the performance figures of the reference runs. A classifier call takes 2.7–3.1 s p50 regardless of evidence; retrieval adds 9–20 ms through the graph and 190–730 ms through the hybrid path, which needs a query embedding round trip and a CPU cross-encoder pass; throughput at the classifier’s concurrency of eight is 2.3–2.7 items per second. Cost is USD 0.21–0.30 per 1,000 items (540 input and 89 output tokens without evidence, 660–990 and 86–105 with). The knowledge base is 108,376 chunks with 1,536-dimensional vectors (660 MB in memory, loaded once per process); the Postgres backend serves the same vectors through an HNSW index for a shared deployment, and the per-company filters keep the dense candidate set small. Judging all 374 promises three times costs

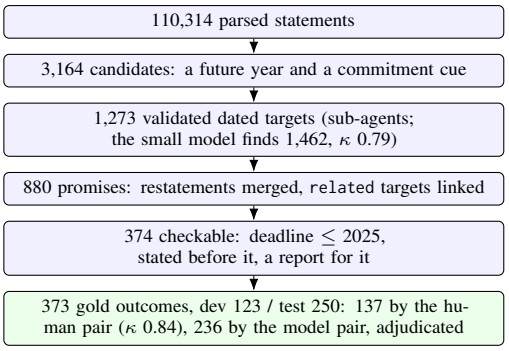


Figure 4: The promise pipeline from statements to the gold set.

USD 1.5–1.9; the whole RQ1 evaluation of one system on dev, four conditions times three repeats, costs about USD 1.

6 Results for RQ2

6.1 Judge and evidence versions on the human checks

Table 12 scores the first and the final system on their human check sheets, weighted to the pools they were drawn from. Overall accuracy on the reported classes is 0.82 in both; what changed is *where* the system is wrong. The first judge, naming the outcome directly, gave a verdict on 31% of the commitments the annotator could not settle, always in the stricter direction (a later year’s result read as the deadline year’s, a different scope or level taken for this target, an annual target set again read as a moved deadline). With per-statement labels and the rule, on held-out rows nothing was tuned on, that over-commitment fell to 14% (2–33) and the verdicts given are right 85% of the time (74–94); the price is 15% of settled commitments left *unclear*. On the 74 rows the prompt was tuned on, the model’s own outcome in the same reply reaches 0.81 and the rule 0.88–0.92, so the rule adds to the questions, mostly on *revised*. The judge is noisy: 16% of promises do not get the same outcome in three runs, almost always a verdict against unclear, hence the majority vote. The *failed*-versus-*revised* boundary accounted for 8 of the 20 disagreements on the held-out sheet, partly a guide ambiguity of ours; the decision to report both as *failed* (the deadline passed without the target being reported as met) moved raw accuracy by 0.03 only and left one rule for annotators and system alike.

arm (run)	macro-F1 mean ± sd	vote	95% CI	none F1	p	retr. p50	USD/1k
sentence only (B)	0.7451 ± 0.019	0.7397	0.683–0.793	0.766	–	–	0.215
sentence only (G)	0.7450 ± 0.014	0.7498	0.695–0.801	0.753	–	–	0.214
sentence only (P)	0.7517 ± 0.005	0.7580	0.704–0.810	0.739	–	–	0.213
neighbouring sentences (B)	0.7453 ± 0.007	0.7617	0.705–0.816	0.705	0.82	–	0.251
hybrid, same report (B)	0.7380 ± 0.006	0.7442	0.687–0.794	0.690	0.68	734 ms	0.259
hybrid, other years (B)	0.7676 ± 0.004	**0.7767**	0.721–0.828	0.739	0.21	191 ms	0.255
graph, other years (G)	0.7472 ± 0.010	0.7449	0.688–0.799	0.731	0.77	9 ms	0.227
graph + BM25 fused (G)	0.7708 ± 0.006	0.7668	0.713–0.818	0.711	1.00	20 ms	0.253
any arm, test split (669 items)	**[Pending Result]**						

Table 6: RQ1 on the development split (331 items, cited classifier, three uncached repeats). Runs: (B) `baseline/statement-hybrid-cited`, (G) `graph/graph-arms-cited`, (P) `baseline/passage-hybrid-cited`. Macro-F1 over the three action labels; the vote is the majority over repeats with its bootstrap 95% interval; p is the paired McNemar test of the arm's vote against the sentence-only vote of the same run; retrieval latency is p50 per item with three runs sharing one machine; cost at 0.20 / 1.20 USD per million input / output tokens.

arm	label	P	R	F1
sentence only	implemented	0.796	0.860	0.827
	planning	0.740	0.864	0.797
	indeterminate	0.735	0.500	0.595
hybrid, other yrs	implemented	0.805	0.882	0.842
	planning	0.803	0.864	0.832
	indeterminate	0.774	0.569	0.656
graph + BM25	implemented	0.807	0.860	0.833
	planning	0.803	0.864	0.832
	indeterminate	0.719	0.569	0.636

Table 7: Per-label precision, recall and F1 of the majority vote (dev; supports 136 / 66 / 72). The confusion is the same in every arm: a quarter of the *indeterminate* items are read as *implemented* (25, 24 and 23 of 72), the LLM failure mode A3CG reports.

condition	faith.	ans. rel.	ctx. P	ctx. rel.
sentence only	0.964	0.574	–	–
local	0.961	0.573	–	–
hybrid, same report	0.976	0.563	0.497	0.369
hybrid, other years	0.973	0.568	0.371	0.286
judge–human agreement	0.976	–	–	0.831
judge lenient by	–	–	–	1.7×

Table 8: Answer-level RAG metrics (run `baseline/statement-hybrid-cited`, repeat 0, 331 items, LLM-judged) and the human calibration of the judge (186 rows). Answer relevance is comparable across conditions, not against 1.0.

arm	passages	statements	flips	p
hybrid, same report	0.7499	0.7442	$+11/-12$	1.00
hybrid, other years	0.7396	**0.7767**	$+18/-7$	**0.043**
local	0.7509	0.7617	$+9/-6$	0.61
sentence only	0.7580	0.7397	$+5/-7$	0.77

Table 9: Granularity ablation: vote macro-F1 under the passage and the statement store, paired on the 331 items (flips passage → statement, McNemar p). Runs `baseline/passage-hybrid-cited` and `baseline/statement-hybrid-cited`.

6.2 Retrieval arms against the gold set

Table 13 runs `gw-eval -task rq2` on the gold development split: one `promise_window` retrieval per promise with each system, the shared rule judge three times over identical evidence, the vote scored against the gold (runs `graph/promise-window-trajectory`, `baseline/promise-window-hybrid`, snapshot `a6f35618dd6e`). The trajectory finder's evidence and the hybrid baseline's top 20 both put the vote at 97 of 123 right (accuracy 0.789, κ 0.69 and 0.68, macro-F1 0.769 and 0.771); paired, $+4/-4$, $p = 1.00$. **On RQ2 as on RQ1, the graph line's structured retrieval and plain hybrid retrieval are not distinguishable on dev.** What differs is how they get there: the finder reads 11.7 statements where the baseline reads 20, so its judgements cost 20% less (USD 1.30 against 1.62 per 1,000) and are steadier (10 promises change label across repeats against 20); the baseline is more cautious (a verdict on 3 of the 22 gold-unclear promises against 5, 21 of 101 settled ones left unclear against 17, 94% of its verdicts right against 90%). Per class (finder): precision / recall achieved 0.93 / 0.82, failed 0.92 / 0.71, unclear 0.50 / 0.77, disappeared 0.67 / 1.00; the confusions are *achieved* and *failed* promises left unclear (10 and 7) and the two boundaries the human annotators also split on, unclear against disappeared when the evidence touches the subject but not the target, and whether a practice shown running counts as achieved. Accuracy on the 22 human-labelled dev rows is 0.68 (both arms) against 0.81 on the model-labelled rows; the human rows are the

condition	macro-F1 mean ± sd	vote	none F1	flips vs S-only	p	spans/item	items citing	USD/1k
`sentence_only`	0.7561 ± 0.015	0.7686	0.804	–	–	0	–	0.184
`local`	0.7598 ± 0.026	0.7745	0.848	$+21/-14$	0.31	–	–	0.226
`same_report_rrf`	0.7544 ± 0.002	0.7528	0.837	$+15/-14$	1.00	2.00	10.5%	0.245
`cross_year_rrf`	0.7841 ± 0.010	**0.7988**	0.787	$+19/-12$	0.28	2.66	1.3%	0.242
`cross_year_agentic`	0.7544 ± 0.002	0.7609	0.792	$+8/-10$	0.82	0.11	7.4%	0.432
`company_agentic`	0.7624 ± 0.011	0.7645	0.828	$+11/-8$	0.65	0.18	23.5%	0.445
any arm, test split	**[Pending Result]** (RQ1 not run on test)							

Table 10: The agentic system on dev, six arms in one run (`agentic/rq1-dev-all-arms`, statement store, cited classifier, three uncached repeats). Flips and p are McNemar of the majority votes against the run's own `sentence_only`; "items citing evidence" is the share of the items given evidence whose rationale cites some of it; the agentic cost includes the planner's tokens.

condition	retr. p50/p95	e2e p50/p95	items/s	USD/1k
sentence only	–	2766/5766	2.51	0.215
local	–	2751/4585	2.61	0.251
hybrid, same report	734/1449	3805/6213	2.29	0.259
hybrid, other years	191/954	3214/6686	2.39	0.255
graph, other years	9/11	2991/5207	2.41	0.227
graph + BM25 fused	20/88	3152/4918	2.39	0.253

Table 11: Latency (ms per item), throughput and cost (runs `baseline/statement-hybrid-cited` and `graph/graph-arms-cited`; three runs shared one machine).

	first system: judge v1, top-6, 75 of 374	final system: rule judge, trajectory, 60 held out of 299
genuine commitment / fields right	0.99 / 0.91	1.00 / 0.82
accuracy, reported classes	0.82 (0.72–0.90)	0.82 (0.70–0.92)
verdict where the check says unclear	31% (20–42)	**14% (2–33)**
unclear where the check has a verdict	0%	15% (3–29)
verdicts that are right	75% (63–86)	85% (74–94)
statements shown per promise	6	11.6

Table 12: The two human check sheets, weighted to their pools, 95% bootstrap intervals (`promise-check-first-evidence.xlsx`, `promise-check-trajectory.xlsx`). The final system's rows were not used for tuning.

earlier, harder sample.

The agentic arms and the noise floor. The agentic system's runs (Table 13, the H′, F and A rows, one batch paired among themselves) place the hybrid baseline at 0.805, a fixed budget per report year with no planner (the finder's rules over the statement store, without the graph's neighbours and links) at 0.813 ($+7/-6$ against hybrid, $p = 1.00$) and the planner on top of it at 0.789 ($+4/-6$ against hybrid, $p = 0.75$; $+2/-5$ against the fixed budget, $p = 0.45$). The same hybrid configuration scored 0.789 on one day and 0.805 on another, which is the noise to read every RQ2 difference against. **Retrieval is not what limits RQ2 on this store**: of the hybrid run's 24 errors, 17 could be checked by matching a phrase from the annotators' notes against the evidence shown, and in 14 of those the phrase was already in front of the judge; doubling the evidence to 40 statements lowers the vote to 0.772 ($+3/-7$, $p = 0.34$) and raises the settled promises left unclear from 16 to 20. The planner mostly stops after one call (1.30 per promise), and of the 195 statements its own actions added, the judge gave a result label to one.

Judge v3 on dev. Re-judging the same stored evidence with v3 (Section 4.8) moves every arm up: fixed budget 0.813 → 0.870 ($+9/-2$, $p = 0.065$), agent 0.789 → 0.837 ($+9/-3$, $p = 0.15$), hybrid 0.805 → 0.813 ($+6/-5$, $p = 1.00$), the trajectory finder 0.789 → 0.837 ($+8/-2$, $p = 0.109$). What changes is under-commitment: settled promises left unclear fall (fixed budget 16 → 8 of 101, finder 17 → 10) at about the same over-commitment and verdict precision. Each added label helps a little when the rule is re-run on the saved labels, `proxy` most (fixed budget: no feature 0.789, `proxy` alone 0.846, all three 0.870). Over eight v3 repeats on the fixed budget's evidence the mean is about 0.85 and single repeats range 0.829–0.870, so 0.870 is the top of the noise; v3 was designed after reading this split's errors and the annotators' notes, and only the test split can say how much of the gain holds. Under v3 the planner still adds nothing (agent against fixed budget $+2/-6$, $p = 0.29$), and keeping only the added statements the judge's labels used (a filter that reads no gold, at the cost of a second round of judging) lifts the agent's evidence to 0.862, level with the fixed budget ($p = 1.00$) and still with a planner to pay for.

arm (run)	judge	vote acc.	κ	macro-F1	over-commit	under-commit	verdict prec.	stmts	USD / 1k promises
trajectory finder, graph (T)	v2	0.789	0.69	0.769	5 / 22	17 / 101	80 / 89 (0.90)	11.7	3.90
trajectory finder, graph (T3)	v3	0.837	0.75	0.788	5 / 22	10 / 101	86 / 96 (0.90)	11.7	4.50
hybrid top-20 in window (H)	v2	0.789	0.68	0.771	3 / 22	21 / 101	78 / 83 (0.94)	20.0	4.86
hybrid top-20, second run (H′)	v2	0.805	0.71	0.788	5 / 22	16 / 101	82 / 90 (0.91)	20.0	5.02
hybrid top-20 (H′ evidence)	v3	0.813	0.71	0.770	6 / 22	12 / 101	84 / 95 (0.88)	20.0	5.64
fixed budget, finder rules without the graph (F)	v2	0.813	0.72	0.798	3 / 22	16 / 101	81 / 88 (0.92)	11.6	3.97
fixed budget (F evidence)	v3	**0.870**	0.80	0.826	4 / 22	8 / 101	89 / 97 (0.92)	11.6	4.48
agent, two hops (A)	v2	0.789	0.69	0.761	3 / 22	21 / 101	78 / 83 (0.94)	14.3	5.14
agent, two hops (A evidence)	v3	0.837	0.75	0.773	5 / 22	11 / 101	86 / 95 (0.91)	14.3	5.68
agent + filter, judged twice (A_f)	v3	0.862	0.79	0.813	5 / 22	9 / 101	89 / 97 (0.92)	11.7	10.15

Table 13: RQ2 on the gold development split (123 promises, three uncached judge repeats, majority vote). Runs: T `graph/promise-window-trajectory`, T3 `graph/promise-window-trajectory-v3` (judge v3 on T's stored evidence), H `baseline/promise-window-hybrid` (2026-10-30), H′ `agentic/rq2-dev-hybrid-k20` (2026-10-31, the same configuration), F `agentic/rq2-dev-fixed-budget`, A `agentic/rq2-dev-two-hops`, the v3 rows `agentic/rq2-dev-v3-*` on the same evidence. Over-commit: a verdict where the gold is unclear; under-commit: unclear where the gold has a verdict. Cost per 1,000 promises at three judgements each, the planner once. Judge v3 was designed on this split's errors, so its rows are optimistic.

6.3 On the test split

The RQ2 test split (250 promises: achieved 145, failed 34, unclear 64, disappeared 7) was read on 2026-10-31 for the agentic system's arms, with the configuration, the pipeline the project chose on the development numbers (agent, two hops; judge v3; the added statements filtered by what v3's labels used; judge v3 again) and eight comparisons frozen and committed before the first test label was read: T1, the chosen pipeline against the hybrid baseline with judge v2; T2, against the fixed budget with v3; T3, against the agent with v3 and no filter; T4 and T5, v2 against v3 on the fixed budget's and the hybrid's evidence; Q1–Q3, the three retrieval arms against each other under v2. The trajectory finder, unchanged since 2026-10-03, and a second run of the hybrid baseline were read on 2026-11-04 so that the graph line is paired on the same promises; the finder's evidence was judged with v2 and, without new retrieval, with v3. Table 14 gives every arm; Table 15 the comparisons.

The graph line on test. Under judge v2 the trajectory finder's vote is 0.736 (184 of 250, κ 0.57) and the second run of the hybrid baseline, on the same machine the same day, 0.708 (177, κ 0.53); paired, $+20/-13$ in the finder's favour, $p = 0.30$ (G1). Both are more cautious on test than on dev: the finder leaves 51 of the 186 settled promises unclear (27%, against 17% on dev), reads all seven *disappeared* promises as unclear, and its verdicts are right 90% of the time (131 of 146) against 85% for the baseline (131 of 155); it reads 11.6 statements at 220 ms where the baseline reads 20 at 932 ms, for USD 4.00 against 4.98 per 1,000 promises. The two batches of test runs, made on different machines on different days, are not interchangeable: the same hybrid configuration scored 0.768 on 2026-10-31 and 0.708 here, a gap of 15 promises, larger than the run-to-run spread seen on dev (0.016), so rows are paired within a batch only. The finder's and the baseline's stored test evidence re-judged with v3 (G2–G4) is **[Pending Result]**: both runs were stopped by memory pressure on the machine after two of three repeats and are to be re-run.

No frozen comparison reaches 0.05, before any correction for making eight. The chosen pipeline is not shown to beat the baseline (T1, $+28/-18$, $p = 0.18$) and costs twice as much; it ties the fixed budget with v3 (T2) at more than twice that cost; and its filter, which looked useful on dev, goes the wrong way on test (T3, $+5/-13$, $p = 0.096$). Judge v3 is the one change in the same direction on every arm, 11 to 14 more promises right of 250 (T4 $+23/-12$, $p = 0.090$; T5 $+24/-12$, $p = 0.065$; on dev the finder $+8/-2$, $p = 0.11$; G3 and G4 on test **[Pending Result]**), at the price of more verdicts where the gold is unclear (hybrid 15 → 20 of 64, fixed budget 17 → 21) and verdicts about as often right. That gain sits on the rows the model annotators labelled: on the group's own 115 rows hybrid goes 87 → 88, the fixed budget 92 → 91 and the agent 89 → 95; on the 135 model-labelled rows 105 → 116 and 99 → 111, and v3's `proxy` reading may share the model annotators' habit. The retrieval arms stay indistinguishable under either judge (Q1–Q3; G1 $+20/-13$ for the finder over the baseline, $p = 0.30$). Dev flattered every v3 figure as expected (fixed budget 0.870 on dev, 0.808 on test), and the agent's order against the fixed budget re-

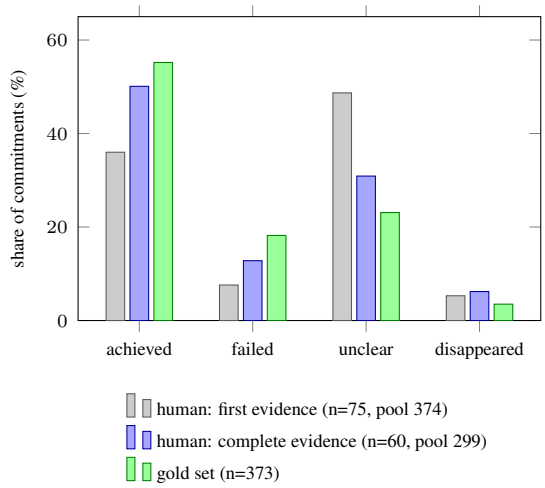


Figure 5: What dated commitments became, by source of the label. The two human-verified bars are different samples weighted to different pools; the gold bar counts every checkable promise.

versed between the splits, so that difference is not to be trusted on either. The unfiltered agent with v3 is the best of the arms read, 0.840, $+32/-14$ against the baseline ($p = 0.011$ uncorrected, about 0.23 after correcting for the arms compared); it was not the chosen pipeline and was picked after reading test, so it is a lead rather than a finding, and test is now spent for RQ2.

6.4 What the commitments did

Figure 5 gives the outcome shares three ways: human-verified on the first evidence (49% unclear), human-verified on the complete evidence (31% unclear), and the gold set over all 373. Where the outcome can be told, four in five commitments were met and one in five was missed or re-dated; a few vanish from the reports. About a third of the first-round *unclear* was the retrieval, not the reports. System counts over the 374 on trajectory evidence: achieved 169, failed 56, unclear 134, disappeared 15; on the hybrid baseline's evidence 172 / 62 / 125 / 15.

6.5 Case studies

Ten commitments from eight companies were followed through every statement of the company, one per pattern (Table 16); none is a finding of greenwashing. Reading the whole timeline changed the sheet's label for five of the ten, and of the eleven commitments the first check called *disappeared*, five were in the reports after all (a partial delivery on page 169 of a 1,976-statement report, a miss stated in a flattened target table, a delay disclosed two years before the deadline). One timeline shows what only the sequence reveals: Gallant Venture wrote in 2022 "We will be expanding our reporting scope to include Scope 3 categories by FY2023" (p.34); in 2023 "We are looking into incorporating several Scope 3 GHG emission categories" (p.26); in 2024 and 2025 "exploring" (p.27). Each later sentence alone is an unremarkable statement of intent and would be labelled *indeterminate*; only the sequence shows a commitment that was dated once and whose date was never replaced. Query Q1 of Table 4 is this case, and the dense ranking returns exactly these sentences.

7 Discussions

Evidence does not change the label, and that is a result. The headline experiment was designed to be falsifiable: evidence should help the ellipsis and attribution cases and leave modality and negation alone. What we measure is a small, consistent, insignificant gain concentrated in *indeterminate* recall, a cost in *none* detection, and a classifier that cites what it is given in 1–9% of items. The retrieved statements are mostly not relevant (corrected context relevance about 0.2), and when they are, the sentence already carried the answer. For an A3CG-style label, context is rarely the missing ingredient; the hard errors are pragmatic (hedging read as implementation), which no amount of same-company text fixes. A larger k, or evidence selected for the specific reference it resolves, might move the needle; a fixed budget of three spans does not.

The graph's value is structure and speed, not accuracy. On RQ1 the claim graph matches hybrid retrieval at a tenth of the latency; on RQ2 its trajectory finder matches the hybrid baseline with 40% fewer statements, cheaper and steadier judgements, and an ordered reading a person can follow. It did not deliver what the proposal hoped for, a better cross-year join through canonical aspects, because canonicalisation failed its precision gate and 79% of extracted aspects stayed singletons; cross-year edges rest on exact normalised matches. The promise graph (targets, restatements, `related` links) is the part of the structure that earned its keep.

The unit and the filter. Two decisions about the knowledge base mattered more than any retriever: retrieving statements rather than passages, and

arm (run)	judge	right	vote acc. (95% CI)	κ	macro-F1	over-commit	under-commit	verdict prec.	stmts	USD/1k
hybrid top-20 (HT)	v2	192	0.768 (0.712–0.816)	0.62	0.637	15 / 64	38 / 186	143 / 163 (0.88)	20.0	5.02
hybrid top-20, 2nd run (HT′)	v2	177	0.708 (0.652–0.764)	0.53	0.597	18 / 64	49 / 186	131 / 155 (0.85)	20.0	4.98
hybrid top-20 (HT evidence)	v3	204	0.816 (0.768–0.860)	0.68	0.669	20 / 64	23 / 186	160 / 183 (0.87)	20.0	5.71
hybrid top-20 (HT′ evidence)	v3	**[Pending Result]** (re-judge stopped by memory pressure after two of three repeats; to be re-run)								
finder, graph (GT)	v2	184	0.736 (0.680–0.792)	0.57	0.548	11 / 64	51 / 186	131 / 146 (0.90)	11.6	4.00
finder (GT evidence)	v3	**[Pending Result]** (same)								
fixed budget (FT)	v2	191	0.764 (0.712–0.816)	0.61	0.684	17 / 64	39 / 186	144 / 164 (0.88)	11.5	3.98
fixed budget (FT evidence)	v3	202	0.808 (0.756–0.856)	0.68	0.687	21 / 64	25 / 186	159 / 182 (0.87)	11.5	4.52
agent, two hops (AT)	v2	196	0.784 (0.732–0.836)	0.64	0.641	12 / 64	40 / 186	144 / 158 (0.91)	14.0	5.13
agent, two hops (AT evidence)	v3	**210**	**0.840** (0.792–0.884)	0.72	0.695	18 / 64	20 / 186	164 / 184 (0.89)	14.0	5.72
chosen pipeline (agent, filter, v3 twice)	v3	202	0.808 (0.760–0.852)	0.67	0.691	22 / 64	23 / 186	160 / 185 (0.87)	11.5	10.26

Table 14: RQ2 on the test split (250 promises, read once per arm; three uncached judge repeats, majority vote). Runs: HT, FT, AT and their v3 rows `agentic/rq2-test-*` (2026-10-31); HT′ `baseline/promise-window-hybrid-test`, GT `graph/promise-window-trajectory-test` and their v3 rows (2026-11-04). The v3 rows re-judge the stored evidence of the v2 row they follow. Cost per 1,000 promises at three judgements, the planner once; HT′ and GT ran on one machine at the same time.

	comparison	right	flips	p
T1	HT v2 → chosen pipeline	192 → 202	+28/−18	0.18
T2	FT v3 → chosen pipeline	202 → 202	+5/−5	1.00
T3	AT v3 → chosen pipeline	210 → 202	+5/−13	0.096
T4	FT, v2 → v3	191 → 202	+23/−12	0.090
T5	HT, v2 → v3	192 → 204	+24/−12	0.065
Q1	HT → AT, v2	192 → 196	+16/−12	0.57
Q2	HT → FT, v2	192 → 191	+19/−20	1.00
Q3	FT → AT, v2	191 → 196	+18/−13	0.47
G1	HT′ → GT, v2	177 → 184	+20/−13	0.30
G2	HT′ → GT, v3	**[Pending Result]**		
G3	GT, v2 → v3	**[Pending Result]**		
G4	HT′, v2 → v3	**[Pending Result]**		
post hoc	HT v2 → AT v3	192 → 210	+32/−14	0.011

Table 15: Paired comparisons on the test split (majority votes, McNemar, uncorrected). T1–T5 and Q1–Q3 were frozen before test was read; G1–G4 pair the graph line and the second baseline run read on 2026-11-04; the last row was chosen after reading test.

company	commitment (deadline)	pattern
Tiong Woon	solar panels (FY2023)	delivered on time, output reported after
Indofood Agri	100% allergen disclosure (2022)	99% at the deadline, delivered a year late, disclosed
Gallant Venture	GHG −10% vs 2017 (2022)	deadline moved to 2024 without comment; level met
Indofood Agri	100% ISPO certification (2023)	moved to 2024, then 2026, with reasons; 89% for three years
Gallant Venture	report Scope 3 (FY2023)	dated commitment dissolves into intent
CDL	SBTi target (FY2024)	milestone slips a year per report
HTrust	electricity per sq ft (annual)	two years met, two missed and said so
Old Chang Kee	zero corruption cases; ≤ 3 lost-time incidents	one issuer, both outcomes, in its own target tables
Kim Heng	solar rooftops at four sites (2023)	two reported, not reconciled
ThaiBev	digital name cards (FY2022)	never mentioned again
LHN		

Table 16: The ten page-cited case studies (full timelines in `docs/case-studies.md`).

ranking *every* statement rather than the sentence-shaped ones. The second is where a sensible-looking filter did real damage, silently, through rank fusion; the human-verified *unclear* share of commitments fell from 49% to 31% once it was removed. Chunking is a design choice with a measurable cost, which is the course's point.

Agentic retrieval. On RQ1 the gate finds a gap in one sentence in five and the verified passages it hands over are cited four times as often as plain retrieval's, but the label does not move and the gate's call on every item doubles the cost, matching Ferrazzi et al. (2026). On RQ2, where finding a result line two reports later looked like a multi-hop plan, the planner adds 2–3 statements per promise that the judge almost never uses (one result label among 195 added statements on dev) and no accuracy on dev or test under either judge; the fixed budget per report year, which is the trajectory finder's reading order without the graph, does as well as anything on the fewest statements and without a model call. An audit of the baseline's dev errors shows why: in 14 of the 17 errors that could be checked the answer phrase was already in front of the judge, and doubling the evidence lowered accuracy. On this store retrieval is not what limits RQ2; the judge is.

Judging outcomes: decompose, rule, abstain. Asking per-statement questions and letting a rule name the outcome halved the verdicts given on commitments a reader cannot settle; the rule's cost is abstention on 15% of settled ones. Both directions are reported, because a user of the output needs the precision of the verdicts (0.90–0.94) more than the accuracy. Judge v3 moves the other way: by accepting the company's own account in other words, a stated part delivery and a figure compared in code, it reads 11 to 14 more promises of 250 through on every arm, at more verdicts where the gold is unclear, and its gain on test sits on the model-annotated rows while the group's own 115 rows barely move; it was designed on the development errors and has no human check, so it is reported as a lead, with v2 as the system of record. The judge is noisy (16% unstable, more on test), tables read across columns are the recurring error, and a bare "this target" is mis-attributed; the model annotators show the same two boundary cases as the humans.

Annotation. The 1,000-sentence set meets the brief's agreement floor on the shared pilot (0.861 observed, κ 0.81), but 950 rows carry one label; the RQ2 gold rests on two humans for 137 rows and on calibrated model annotators of two families for 236, with the calibration reported. Model annotators sit where the humans sit against each other, and two instances of one model are not a second opinion. We report these as limits, not as equivalents of a fully double-annotated set.

Limits and what is not done. RQ1's test split is unread; RQ2's has been read once per arm and is spent, so nothing changed from here on can be checked on it; the two batches of RQ2 test runs were made on different machines on different days and are paired within a batch only; the v3 re-judge of the graph line's test evidence is pending; rank-aware retrieval metrics need pooled judgements that were not collected; the demo front end runs single development items and is not a source of numbers; a stock-price analysis was screened in the literature and judged under-powered at 28 firms, so it is future work; retrieval latencies were measured with three runs sharing one machine; extraction misses targets that exist only inside tables and scope that the promise sentence left to its context (fields right 0.82 on the stricter check). Everything is assistive screening: a postponement can have a good reason, and an unmentioned pledge may have been kept.

8 Conclusion

We built a retrieval-augmented system over 240 SGX sustainability reports with a knowledge base of every parsed statement, a hybrid retriever, a temporal claim graph, an agentic planner and one shared, cited classifier, and evaluated it on a self-annotated set of 1,000 sentences and a gold set of 373 dated commitments with paired tests throughout. Five things can be claimed on the development splits. Statement-level evidence from the company's own reports neither helps nor hurts action classification beyond run-to-run noise, and passages from other years are worse than statements. A temporal claim graph retrieves cross-year evidence at the same accuracy as hybrid retrieval at 9–20 ms against 190–730 ms. Following a dated commitment through later reports is feasible: on the test split the trajectory finder's verdicts are right 90% of the time (0.736 accuracy under the judge of record, $+20/-13$ against the hybrid baseline, $p = 0.30$), and it over-commits on 17–23% of the commitments a reader cannot settle. Letting a model plan the retrieval is not shown to help on either question and costs more; the lever on RQ2 is the judge, where a looser reading of "this commitment" gains 11 to 14 promises of 250 on every arm, short of significance and mainly on model-annotated rows. About half of the dated commitments in these reports can be verified as met from the issuer's later reports, one in six to one in eight was missed or redated, a quarter to a third cannot be judged from the text, and a few vanish. Checking outcomes needs every statement in the index, table rows and result lines included. RQ1's test split, the v3 re-judge of the graph line's test evidence and the retrieval qrels are pending; every number above carries the run it comes from.

References

Neil Heinrich Braun, Keane Ong, Rui Mao, Erik Cambria, and Gianmarco Mengaldo. 2026. Enhancing language models for robust greenwashing detection. *Preprint*, arXiv:2601.21722.

Tom Calamai, Oana Balalau, Théo Le Guenedal, and Fabian M. Suchanek. 2025. Detecting greenwashing: A natural language processing literature survey. *Preprint*, arXiv:2502.07541.

Jifan Chen, Aniruddh Sriram, Eunsol Choi, and Greg Durrett. 2022. Generating literal and implied subquestions to fact-check complex claims. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 3495–3516, Abu Dhabi, United Arab Emirates. Association for Computational Linguistics.

Jacob Cohen. 1960. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. 2009. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759.

Thomas G. Dietterich. 1998. Approximate statistical tests for comparing supervised classification learning algorithms. *Neural Computation*, 10(7):1895–1923.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2024. From local to global: A graph RAG approach to query-focused summarization. *Preprint*, arXiv:2404.16130.

Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta. Association for Computational Linguistics.

Pietro Ferrazzi, Milica Cvjeticanin, Alessio Piraccini, and Davide Giannuzzi. 2026. Is agentic RAG worth it? an experimental comparison of RAG approaches. *Preprint*, arXiv:2601.07711. Accepted at ACL 2026, Industry Track.

Joseph L. Fleiss. 1971. Measuring nominal scale agreement among many raters. *Psychological Bulletin*, 76(5):378–382.

Johannes Florstedt, Jonas Fahlbusch, and Moritz Sontheimer. 2025. Detecting greenwashing in ESG reports: A comparative analysis of machine learning methods in traffic-related emissions disclosure. In *Proceedings of the 10th edition of the Swiss Text Analytics Conference*, pages 25–30, Winterthur, Switzerland. Association for Computational Linguistics.

Sherine George and Nithish Saji. 2025. ESGBench: A benchmark for explainable ESG question answering in corporate sustainability reports. *Preprint*, arXiv:2511.16438. AI4DF workshop at ACM ICAIF 2025.

Shuang Jiang, Sehoon Kim, and Yucheng Lu. 2025. Limited accountability and awareness of corporate emissions target outcomes. *Nature Climate Change*, 15:279–286. VERIFY title, author initials and page range against the published article.

Georgios Kaoukis, Ioannis Aris Koufopoulos, Eleni Psaroudaki, Danae Pla Karidi, Evaggelia Pitoura, George Papastefanatos, and Panayiotis Tsaparas. 2025. EmeraldMind: A knowledge graph-augmented framework for greenwashing detection. *Preprint*, arXiv:2512.11506. To appear in The ACM Web Conference 2026.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33*, pages 9459–9474.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Keane Ong, Rui Mao, Deeksha Varshney, Erik Cambria, and Gianmarco Mengaldo. 2025. Towards robust ESG analysis against greenwashing risks: Aspect-action analysis with cross-category generalization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14854–14879, Vienna, Austria. Association for Computational Linguistics.

Ralph Peeters, Aaron Steiner, and Christian Bizer. 2025. Entity matching using large language models. In *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*. ArXiv:2310.11244.

Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, Hong Kong, China. Association for Computational Linguistics.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Yohei Seki, Hakusen Shu, Anaïs Lhuissier, Hanwool Lee, Juyeon Kang, Min-Yuh Day, and Chung-Chi Chen. 2025. ML-Promise: A multilingual dataset for corporate promise verification. In *Proceedings of the 2025 Conference on Empirical Methods in Natural*

Language Processing, pages 20353–20366, Suzhou, China. Association for Computational Linguistics.

James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: a large-scale dataset for fact extraction and VERification. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 809–819, New Orleans, Louisiana. Association for Computational Linguistics.

Shikhar Vashishth, Prince Jain, and Partha Talukdar. 2018. CESI: Canonicalizing open knowledge bases using embeddings and side information. In *Proceedings of the 2018 World Wide Web Conference*, pages 1317–1327.

Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems 33*, pages 5776–5788.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Bingbing Wen, Jihan Yao, Shangbin Feng, Chenjun Xu, Yulia Tsvetkov, Bill Howe, and Lucy Lu Wang. 2025. Know your limits: A survey of abstention in large language models. *Transactions of the Association for Computational Linguistics*, 13:529–556. VERIFY volume and pages; arXiv:2407.18418.

Bowen Zhang and Harold Soh. 2024. Extract, define, canonicalize: An LLM-based framework for knowledge graph construction. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9820–9836, Miami, Florida, USA. Association for Computational Linguistics.

Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, Runyuan He, Yongwei Zhang, Sicong Liang, Xilin Liu, Yuchi Ma, and Yixiang Fang. 2025. In-depth analysis of graph-based RAG in a unified framework. *Preprint*, arXiv:2503.04338.

A Reproduction

```
uv sync && cp .env.example .env     # keys
unzip gw-kb-statements-<hash>.zip \
      -d artifacts/baseline/
uv run gw-baseline stats           # d8a62bfe40c6
uv run pytest -q                    # 94 tests
# RQ1, one system, dev, three repeats
uv run gw-eval --system baseline-rag \
  --split dev --repeats 3 --conditions \
  sentence_only,local,same_report_rrf,\
  cross_year_rrf
uv run gw-eval --system graph-rag --split dev \
  --repeats 3 --conditions \
  sentence_only,local,cross_year_graph,\
  cross_year_fused
# RQ2, any system, shared judge
uv run gw-eval --task rq2 --system graph-rag \
  --split dev --repeats 3
uv run gw-eval --task rq2 --system baseline-rag \
  --split dev --repeats 3
# publish, index, compare (paired McNemar)
uv run gw-eval --publish artifacts/runs/<id> \
  --name <group>/<name> --note "..."
uv run gw-eval --compare results/<a> results/<b>
uv run gw-eval --ragas artifacts/runs/<id>
# serve the API; the demo front end
uv run gw-serve --systems baseline-rag,graph-rag
uv sync --group frontend
uv run streamlit run frontend/app.py
```

The promise pipeline is the `gw-graph` subcommands `targets-export`, `targets-ingest`, `promises`, `promise-evidence`, `promise-outcomes`, `promise-sheet`, `promise-score` and `promise-gold`; the graph build is `gw-graph build`. Both are documented in `docs/promises.md` and `docs/graph.md`; every published run is under `results/` with its manifest.